

FINWEALTH GLOBAL

Wealth Analytics Data Platform

End-to-End Data Engineering Project Report

Built on Microsoft Azure Data Factory

Platform	Microsoft Azure
ETL Engine	Azure Data Factory (ADF)
Database	Azure SQL Database
Storage	Azure Blob Storage
Version Control	GitHub
Reporting Layer	Microsoft Power BI
Project Type	End-to-End Data Engineering
Document Type	Technical Project Report

Table of Contents

Abbreviations and Technologies Used	5
Abbreviations	5
Technologies Used.....	5
1. Project Overview	6
1.1 Business Context and Objectives	6
1.2 High-Level Architecture	6
1.3 Source Files Summary	6
1.4 End-to-End Data Flow	7
Stage 1 — Source to Blob Storage	7
Stage 2 — Blob Storage to Staging (pl_ingest_AllSources)	7
Stage 3 — Staging to DWH (pl_transform_stg_to_dwh).....	7
Stage 4 — Master Pipeline Orchestration (pl_masterpipeline)	7
2. Azure Resources Setup	8
2.1 Resource Group	8
2.2 Azure SQL Server and Database	8
2.3 Azure Storage Account.....	9
2.4 Azure Key Vault.....	9
2.5 Azure Data Factory Instance	9
2.6 Azure Resource Inventory	9
2.7 ADF Component Inventory	10
3. Database Design and Schema	11
3.1 Schema Architecture	11
3.2 Null Handling Strategy	12
4. Staging Layer — Table Definitions	14
4.1 Staging Tables Summary	14
4.2 Staging Table DDL Scripts	15
STG-1: stg.stg_advisors	15
STG-2: stg.stg_clients	15
STG-3: stg.stg_accounts	15
STG-4: stg.stg_securities	15
STG-5: stg.stg_benchmarks.....	16
STG-6: stg.stg_cashflows	16
STG-7: stg.stg_fees.....	16
STG-8: stg.stg_holdings	16
STG-9: stg.stg_performance	17
STG-10: stg.stg_prices.....	17

STG-11: stg.stg_trades	17
5. Data Warehouse Schema — Dimension and Fact Tables	18
5.1 Dimension Tables.....	18
5.2 Dimension Table DDL Scripts.....	18
DIM-1: dwh.dim_advisor (SCD Type 1).....	18
DIM-2: dwh.dim_client (SCD Type 2).....	19
DIM-3: dwh.dim_account (SCD Type 2)	19
DIM-4: dwh.dim_security (SCD Type 1).....	20
DIM-5: dwh.dim_benchmark (SCD Type 1)	20
DIM-6: dwh.dim_date (Static Date Dimension)	21
5.3 Fact Tables	21
FACT-1: dwh.fact_holding	22
FACT-2: dwh.fact_trade	23
FACT-3: dwh.fact_cashflow	23
FACT-4: dwh.fact_fee	24
FACT-5: dwh.fact_security_price	24
FACT-6: dwh.fact_performance	24
5.4 Audit Table	25
6. Azure Data Factory Pipeline — Configuration and Design	26
6.1 Linked Services	26
6.2 Datasets	26
BLOB Source Datasets (ds_blob folder)	27
SQL Staging Datasets (ds_sql folder)	27
6.3 Pipeline 1 — pl_ingest_AllSources	28
6.4 Pipeline 2 — pl_transform_stg_to_dwh	29
6.5 Pipeline 3 — pl_masterpipeline (Orchestrator)	31
7. Transformation Logic — Stored Procedures	33
7.1 Stored Procedures Summary	33
7.2 Dimension Stored Procedures	34
SP-1: dwh.usp_load_dim_advisor (SCD Type 1).....	34
SP-2: dwh.usp_load_dim_benchmark (SCD Type 1)	34
SP-3: dwh.usp_load_dim_security (SCD Type 1).....	35
SP-4: dwh.usp_load_dim_client (SCD Type 2).....	36
SP-5: dwh.usp_load_dim_account (SCD Type 2).....	37
7.3 Fact Stored Procedures.....	38
SP-6: dwh.usp_load_fact_holding	38
SP-7: dwh.usp_load_fact_trade (with Deduplication Logic).....	39
SP-8: dwh.usp_load_fact_cashflow	40

SP-9: dwh.usp_load_fact_fee 41

SP-10: dwh.usp_load_fact_security_price 41

SP-11: dwh.usp_load_fact_performance 42

8. Pipeline Validation and Final Output Verification 43

8.1 Final Row Count Verification 43

8.2 Data Quality Summary 44

8.2 Referential Integrity Check 45

8.3 Duplicate Trade Check 45

8.4 Audit Log Check 45

9. Analytics Readiness and Power BI Integration..... 46

9.1 Power BI Data Model..... 46

9.2 Supported Analytical Use Cases 46

9.3 GitHub Integration and Version Control 46

10. Conclusion 49

Abbreviations and Technologies Used

Abbreviations

Abbreviation	Full Form
ADF	Azure Data Factory
AUM	Assets Under Management
CSV	Comma Separated Values
DDL	Data Definition Language
DWH	Data Warehouse
ETL	Extract, Transform, Load
FK	Foreign Key
PK	Primary Key
SK	Surrogate Key
SCD	Slowly Changing Dimension
SP	Stored Procedure
SQL	Structured Query Language
STG	Staging

Technologies Used

Category	Technology / Tool	Purpose
Cloud Platform	Microsoft Azure	Host all cloud resources
ETL / Orchestration	Azure Data Factory (ADF)	Build and run data pipelines
Database	Azure SQL Database	Host staging and DWH tables
Storage	Azure Blob Storage	Store source CSV files
Secret Management	Azure Key Vault	Securely store connection strings and credentials
Query Language	SQL (T-SQL)	DDL, stored procedures, validations

Category	Technology / Tool	Purpose
Version Control	GitHub	Store ADF JSON and SQL scripts
Reporting	Microsoft Power BI	Analytics and dashboard layer
IDE / Query Tool	Azure SQL Query Editor / SSMS	Run and validate SQL queries

1. Project Overview

This document provides a complete technical walkthrough of the FinWealth Global Data Engineering pipeline, built using Microsoft Azure Data Factory (ADF). The objective of this project was to design and implement a production-grade, end-to-end ETL pipeline that ingests raw CSV data from Azure Blob Storage, loads it into SQL staging tables, applies rigorous data quality transformations, and populates a clean Star Schema data warehouse, ready for Power BI reporting and advanced analytics.

FinWealth Global is a mid-sized wealth management firm offering portfolio advisory, mutual funds, insurance, and financial planning services to clients across multiple regions. Their data originates from multiple operational systems — a CRM for client and advisor assignments, a transaction system for investments and redemptions, a market data feed for daily NAV pricing, and a commission system for advisor incentives. This project consolidates data from all these sources into a unified, analytics-ready data warehouse on Azure.

1.1 Business Context and Objectives

The primary business objectives driving this data engineering initiative are as follows:

- Enable accurate, consolidated reporting of Assets Under Management (AUM) across all client accounts and securities.
- Provide visibility into advisor performance, measured against benchmark return percentages and excess return metrics.
- Support client profitability analysis by account type, segment, and risk profile.
- Enable product mix analysis across asset classes, sectors, and securities.
- Track trade settlement efficiency and identify settlement lag patterns.
- Deliver analytical-ready datasets to Power BI for executive dashboards and operational reporting.

1.2 High-Level Architecture

The solution follows a medallion-style layered architecture across three processing zones within a single Azure SQL Database instance. Raw data flows from Azure Blob Storage into the staging layer, is validated and transformed through stored procedures, and is loaded into the data warehouse layer from where Power BI consumes it.

Layer	Technology	Purpose
Source	Azure Blob Storage	Holds 11 raw CSV files exported from operational systems
Staging (stg)	Azure SQL Database — stg schema	Raw copy of CSV data with no transformations applied
Data Warehouse (dwh)	Azure SQL Database — dwh schema	Cleaned Star Schema — 5 dimensions, 6 fact tables
Orchestration	Azure Data Factory (ADF)	Pipelines, Copy Activities, Stored Procedure Activities
Error Tracking	dwh.error_log table	Captures rejected rows with rejection reason and timestamp
Audit Logging	dbo.audit_pipeline_log	Pipeline-level run status, row counts, success/failure tracking
Reporting	Microsoft Power BI	End-user dashboards consuming the DWH Star Schema

1.3 Source Files Summary

Eleven CSV source files were provided, representing exports from the firm's operational systems. The table below summarises each file, its row count, any identified data quality issues, and the target destination in the data warehouse.

Source File	Row Count	Data Quality Issues	DWH Destination
Clients.csv	8,000	RiskProfile: 66 null values	dwh.Dim_Client
Advisors.csv	350	Region: 1 null value	dwh.Dim_Advisor
Accounts.csv	12,000	None	dwh.Dim_Account
Securities.csv	3,000	Sector: 672 null values	dwh.Dim_Security
Benchmarks.csv	4,020	None (6 distinct codes)	dwh.Dim_Benchmark
Holdings.csv	96,027	None	dwh.Fact_Holdings
Trades.csv	100,400	400 duplicate rows; 3 amended amounts	dwh.Fact_Trades
Cashflows.csv	30,000	None	dwh.Fact_Cashflows
Fees.csv	18,000	None	dwh.Fact_Fees
Performance.csv	110,000	ReturnPct: 109 null values	dwh.Fact_Performance
Prices.csv	301,500	ClosePrice: 478 null values	dwh.Fact_Security_Price

1.4 End-to-End Data Flow

The data pipeline executes in four sequential stages, each managed and orchestrated by Azure Data Factory. The master pipeline ensures that no stage begins until the preceding stage has completed successfully.

Stage 1 — Source to Blob Storage

All eleven CSV files are uploaded to Azure Blob Storage in a container named raw-data. This is a one-time manual upload for the initial load. The files represent read-only exports from operational systems and are never modified during pipeline execution.

Stage 2 — Blob Storage to Staging (pl_ingest_AllSources)

ADF Pipeline pl_ingest_AllSources reads each CSV file from Blob Storage and loads raw data into the eleven SQL staging tables. A pre-copy TRUNCATE script is executed on each staging table before loading, ensuring a complete and clean reload. All eleven Copy Activities run in parallel for maximum throughput, reducing total ingestion time to approximately four to five minutes.

Stage 3 — Staging to DWH (pl_transform_stg_to_dwh)

ADF Pipeline pl_transform_stg_to_dwh executes eleven stored procedures via Stored Procedure Activities. The five dimension stored procedures run sequentially in dependency order first, ensuring that surrogate keys are generated and available before the six fact stored procedures execute. The six fact SPs then run in parallel for performance. All data quality transformations, null handling, type casting, deduplication, and surrogate key resolution happen within these stored procedures.

Stage 4 — Master Pipeline Orchestration (pl_masterpipeline)

ADF Pipeline pl_masterpipeline orchestrates both child pipelines in the correct sequence using Execute Pipeline activities connected by a success dependency. It triggers pl_ingest_AllSources and waits for completion before triggering pl_transform_stg_to_dwh. If the ingestion pipeline fails for any reason, the transformation pipeline is not triggered, preventing partial or inconsistent data loads.

2. Azure Resources Setup

All Azure resources were provisioned under a single Resource Group to enable unified management, cost tracking, and access control. Resources were deployed in the same Azure region to minimise inter-service latency. The following subsections describe each resource, its configuration, and its role within the overall architecture.

2.1 Resource Group

A Resource Group (rg-finwealth01) acts as the logical container for all Azure resources belonging to this project. By grouping all services together, the project benefits from unified billing, centralised IAM (Identity and Access Management) policies, and simplified lifecycle management. This approach is consistent with Azure best practices for project-scoped resource organisation.

 **rg-finwealth01**
Resource group

Search

◊ ◀

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Events

Settings

Cost Management

Monitoring

Automation

Help

Export resource groups using Bicep or Terraform +2

✕

+ Create

⚙️ Manage view ▾

🗑️ Delete resource group

🔄 Refresh

⋮

📄 Group by none ▾

Essentials

JSON View

Resources

Recommendations

Filter for any field...

Type equals all ✕

Location equals all ✕

+ Add filter

☐	Name ↑	Type	Location
☐	 adf-finwealth01	⋮ Data factory (V2)	Central India
☐	 kv-finwealth01	⋮ Key vault	Central India
☐	 sql-db-finwealth01 (sql-finwealth01-ser	SQL database	Central India
☐	 sql-finwealth01-server	⋮ SQL server	Central India
☐	 stfinwealth01	⋮ Storage account	Central India

2.2 Azure SQL Server and Database

An Azure SQL Server (sql-finwealth01-server) was provisioned as the logical host for the project database. The database (Sql-db-finwealth01) was created on this server and hosts all staging tables (stg schema), data warehouse tables (dwh schema), and audit tables (dbo schema).

SQL Authentication was enabled and the server firewall was configured to allow connections from Azure services and the developer's IP address. Three schemas were created within the database to logically separate the layers of the data architecture:

- stg schema — holds all eleven staging tables that receive raw CSV data from ADF Copy Activities.
- dwh schema — holds all dimension tables, fact tables, the error log, and all transformation stored procedures.
- dbo schema — holds the audit pipeline log table and the audit stored procedures.

2.3 Azure Storage Account

An Azure Storage Account (stfinwealth01) was provisioned to host a Blob Container that stores all eleven source CSV files. The container was configured with private access, ensuring that only authorised services — specifically the ADF Blob linked service — can read the files. The eleven CSV files were uploaded to the container prior to pipeline execution and are treated as read-only inputs throughout the project.

2.4 Azure Key Vault

An Azure Key Vault (kv-finwealth01) was provisioned to securely store sensitive connection strings and credentials used by ADF linked services. Rather than embedding credentials directly into linked service definitions (which would expose them in pipeline JSON and version control), the Key Vault reference method was used. This ensures that secrets are never exposed in pipeline configurations, source code, or repository history — a critical security best practice for enterprise data engineering.

2.5 Azure Data Factory Instance

The Azure Data Factory instance (adf-finwealth01) serves as the central orchestration engine for the entire pipeline. ADF was selected for this project because it provides a fully managed, cloud-native ETL and data integration service that natively integrates with Azure Blob Storage, Azure SQL Database, and Azure Key Vault. It supports visual pipeline authoring, built-in activity monitoring and alerting, and native GitHub integration for version control of all pipeline JSON definitions.

2.6 Azure Resource Inventory

Azure Resource	Resource Name	Purpose
Resource Group	rg-finwealth01	Logical container for all project resources

Azure Resource	Resource Name	Purpose
SQL Server	sql-finwealth01-server	Logical host for the project database
SQL Database	Sql-db-finwealth01	Hosts staging, DWH, and audit tables
Storage Account	stfinwealth01	Blob container for source CSV files
Key Vault	kv-finwealth01	Secure secret storage for credentials
Data Factory	adf-finwealth01	ETL orchestration and pipeline management

2.7 ADF Component Inventory

Component	Count	Details
Linked Services	3	ls_keyvault_finwealth, ls_blob_finwealth_raw, ls_sql_finwealth_dwh
Datasets	22	11 CSV source datasets (ds_csv folder) + 11 SQL staging datasets (ds_sql folder)
Pipelines	3	pl_ingest_AllSources, pl_transform_stg_to_dwh, pl_masterpipeline

3. Database Design and Schema

3.1 Schema Architecture

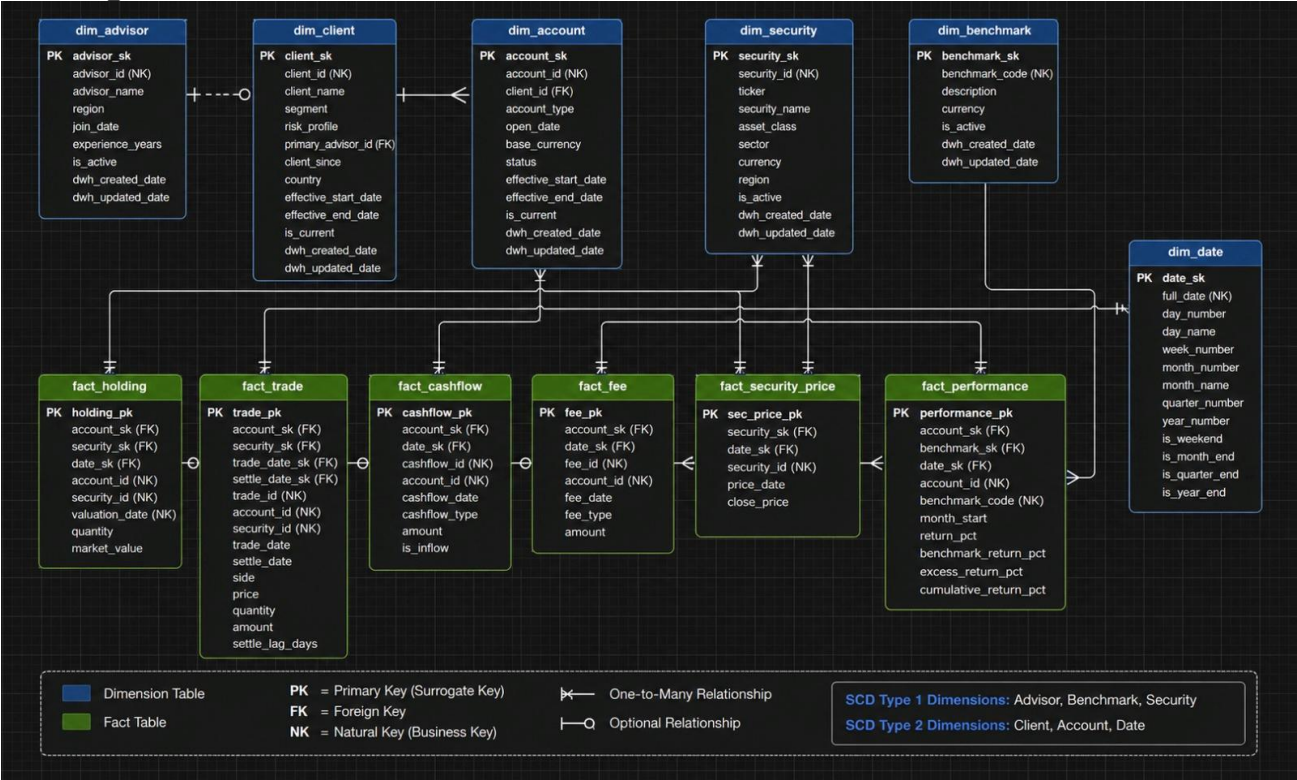
The database follows a two-schema architecture within a single Azure SQL Database instance. The stg schema holds raw staging tables that are structurally identical to the source CSV files — all columns are nullable, no constraints are applied, and no transformations are performed at this layer. The dwh schema holds the cleaned, transformed data warehouse tables organised as a Star Schema.

The Star Schema design was chosen because it optimises analytical query performance through simplified join paths, simplifies Power BI relationship modelling, and is the industry standard for data warehouses supporting business intelligence workloads. The schema comprises five dimension tables and six fact tables connected through surrogate key foreign key relationships.

Schema	Tables	Purpose
stg	11 staging tables	Raw copy of CSV data — no constraints, all columns nullable
dwh	5 dimension tables + 6 fact tables	Cleaned Star Schema data warehouse
dbo	audit_pipeline_log	Pipeline execution audit trail — run status and row counts

Schema Creation Scripts
CREATE SCHEMA stg; CREATE SCHEMA dwh;

ER Diagram



3.2 Null Handling Strategy

A core design decision in this project was how to handle null values present in the source data. The guiding principle was clear: the data engineer's role is to handle whatever the source system provides gracefully and professionally, without modifying the source files and without discarding valid records solely due to missing descriptor fields.

Three distinct null handling approaches were applied, each justified by the nature of the affected column and the business impact of the decision:

Source File	Column	Null Count	Column Type	Decision	Value Applied	Justification
Clients.csv	RiskProfile	66	Descriptor	Default	'Unknown'	The client record is valid without a risk profile. Replacing with 'Unknown' preserves all client data.
Advisors.csv	Region	1	Descriptor	Default	'Unknown'	The advisor record is valid without a region. Dropping this record would cause downstream FK failures.

Source File	Column	Null Count	Column Type	Decision	Value Applied	Justification
Securities.csv	Sector	672	Descriptor	Default	'N/A'	Bond, Cash, and Commodity securities genuinely have no sector classification. 'N/A' is accurate.

Technical Implementation

Null handling is applied within each stored procedure using the T-SQL expression: ISNULL(NULLIF(src.Column, ''), 'DefaultValue'). This pattern handles both true NULL values and empty strings ("), ensuring consistent output regardless of how the source system represents missing values.

4. Staging Layer — Table Definitions

The staging layer contains eleven tables, one for each source CSV file. These tables mirror the source file structure exactly. All columns are defined as VARCHAR to accept data exactly as provided by the source — no type casting, no constraints, and no transformations are applied at this stage. A `stg_load_timestamp` column is added to each staging table to record when the data was loaded, supporting audit and debugging.

Each staging table is fully truncated and reloaded on every pipeline run, ensuring the staging layer always reflects the most current state of the source files. This full-refresh approach eliminates the risk of stale or accumulating data in the staging zone.

4.1 Staging Tables Summary

Staging Table	Source File	Row Count	Key Columns
stg.stg_Advisors	Advisors.csv	350	AdvisorID, AdvisorName, Region, JoinDate, ExperienceYears
stg.stg_Clients	Clients.csv	8,000	ClientID, ClientName, Segment, RiskProfile, PrimaryAdvisorID, Country
stg.stg_Accounts	Accounts.csv	12,000	AccountID, ClientID, AccountType, OpenDate, BaseCurrency, Status
stg.stg_Securities	Securities.csv	3,000	SecurityID, Ticker, SecurityName, AssetClass, Sector, Currency
stg.stg_Benchmarks	Benchmarks.csv	4,020	BenchmarkCode, Description, Currency, IndexValueDate, IndexValue
stg.stg_Holdings	Holdings.csv	96,027	AccountID, SecurityID, Quantity, ValuationDate, MarketValue
stg.stg_Trades	Trades.csv	100,400	TradeID, AccountID, SecurityID, TradeDate, Side, Price, Quantity, Amount
stg.stg_Cashflows	Cashflows.csv	30,000	CashflowID, AccountID, CashflowDate, Type, Amount
stg.stg_Fees	Fees.csv	18,000	FeeID, AccountID, FeeDate, FeeType, Amount
stg.stg_Prices	Prices.csv	301,500	SecurityID, PriceDate, ClosePrice

4.2 Staging Table DDL Scripts

The following DDL scripts define all eleven staging tables. All columns use VARCHAR data types to ensure that raw data from the source CSVs is accepted without type conversion errors at the ingestion stage. Type casting occurs during the transformation stored procedures.

STG-1: stg.stg_advisors

```
CREATE TABLE stg.stg_advisors (  
    AdvisorID          VARCHAR(20),  
    AdvisorName        VARCHAR(100),  
    Region             VARCHAR(50),  
    JoinDate           VARCHAR(20),  
    ExperienceYears     VARCHAR(10),  
    stg_load_timestamp DATETIME DEFAULT GETDATE()  
);
```

STG-2: stg.stg_clients

```
CREATE TABLE stg.stg_clients (  
    ClientID           VARCHAR(20),  
    ClientName         VARCHAR(100),  
    Segment            VARCHAR(50),  
    RiskProfile         VARCHAR(50),  
    PrimaryAdvisorID   VARCHAR(20),  
    ClientSince        VARCHAR(20),  
    Country            VARCHAR(50),  
    stg_load_timestamp DATETIME DEFAULT GETDATE()  
);
```

STG-3: stg.stg_accounts

```
CREATE TABLE stg.stg_accounts (  
    AccountID          VARCHAR(20),  
    ClientID           VARCHAR(20),  
    AccountType        VARCHAR(50),  
    OpenDate           VARCHAR(20),  
    BaseCurrency       VARCHAR(10),  
    Status             VARCHAR(20),  
    stg_load_timestamp DATETIME DEFAULT GETDATE()  
);
```

STG-4: stg.stg_securities

```
CREATE TABLE stg.stg_securities (  
    SecurityID         VARCHAR(20),  
    Ticker             VARCHAR(20),  
    SecurityName       VARCHAR(100),
```

```
AssetClass      VARCHAR(50),
Sector          VARCHAR(50),
Currency        VARCHAR(10),
Region          VARCHAR(50),
stg_load_timestamp DATETIME DEFAULT GETDATE()
);
```

STG-5: stg.stg_benchmarks

```
CREATE TABLE stg.stg_benchmarks (
  BenchmarkCode VARCHAR(20),
  Description    VARCHAR(100),
  Currency       VARCHAR(10),
  IndexValueDate VARCHAR(20),
  IndexValue     VARCHAR(30),
  stg_load_timestamp DATETIME DEFAULT GETDATE()
);
```

STG-6: stg.stg_cashflows

```
CREATE TABLE stg.stg_cashflows (
  CashflowID VARCHAR(20),
  AccountID   VARCHAR(20),
  CashflowDate VARCHAR(20),
  Type        VARCHAR(30),
  Amount      VARCHAR(30),
  stg_load_timestamp DATETIME DEFAULT GETDATE()
);
```

STG-7: stg.stg_fees

```
CREATE TABLE stg.stg_fees (
  FeeID VARCHAR(20),
  AccountID VARCHAR(20),
  FeeDate VARCHAR(20),
  FeeType VARCHAR(50),
  Amount VARCHAR(30),
  stg_load_timestamp DATETIME DEFAULT GETDATE()
);
```

STG-8: stg.stg_holdings

```
CREATE TABLE stg.stg_holdings (
  AccountID VARCHAR(20),
  SecurityID VARCHAR(20),
  Quantity VARCHAR(30),
  ValuationDate VARCHAR(20),
  MarketValue VARCHAR(30),
  stg_load_timestamp DATETIME DEFAULT GETDATE()
);
```

STG-9: stg.stg_performance

```
CREATE TABLE stg.stg_performance (  
    AccountID          VARCHAR(20),  
    MonthStart         VARCHAR(20),  
    ReturnPct          VARCHAR(30),  
    BenchmarkCode      VARCHAR(20),  
    BenchmarkReturnPct VARCHAR(30),  
    ExcessReturnPct    VARCHAR(30),  
    CumulativeReturnPct VARCHAR(30),  
    stg_load_timestamp DATETIME DEFAULT GETDATE()  
);
```

STG-10: stg.stg_prices

```
CREATE TABLE stg.stg_prices (  
    SecurityID  VARCHAR(20),  
    PriceDate   VARCHAR(20),  
    ClosePrice  VARCHAR(30),  
    stg_load_timestamp DATETIME DEFAULT GETDATE()  
);
```

STG-11: stg.stg_trades

```
CREATE TABLE stg.stg_trades (  
    TradeID      VARCHAR(20),  
    AccountID    VARCHAR(20),  
    SecurityID   VARCHAR(20),  
    TradeDate    VARCHAR(20),  
    SettleDate   VARCHAR(20),  
    Side         VARCHAR(10),  
    Price        VARCHAR(30),  
    Quantity     VARCHAR(30),  
    Amount       VARCHAR(30),  
    stg_load_timestamp DATETIME DEFAULT GETDATE()  
);
```

5. Data Warehouse Schema — Dimension and Fact Tables

5.1 Dimension Tables

The data warehouse contains five dimension tables. Each dimension table is assigned a surrogate key implemented as an IDENTITY(1,1) column, which serves as the primary key and decouples the data warehouse from the natural key conventions of the source systems. Natural keys are retained as additional columns to facilitate joins during stored procedure execution and to support data lineage tracing.

Two dimension tables — Dim_Client and Dim_Account — implement Slowly Changing Dimension Type 2 (SCD Type 2) logic, which preserves a full history of changes to tracked attributes. This is critical for regulatory compliance in a wealth management context, where it may be necessary to reconstruct which advisor managed a client at any historical point in time, or what the account status was during a given period.

Dimension Table	Surrogate Key	Natural Key	SCD Type	Row Count	Tracked Attributes
dwh.Dim_Advisor	AdvisorKey	AdvisorID	Type 1	350	AdvisorName, Region, ExperienceYears
dwh.Dim_Client	ClientKey	ClientID	Type 2	8,000	Segment, RiskProfile, Country, PrimaryAdvisorID
dwh.Dim_Account	AccountKey	AccountID	Type 2	12,000	Status, AccountType, BaseCurrency
dwh.Dim_Security	SecurityKey	SecurityID	Type 1	3,000	Ticker, AssetClass, Sector, Currency, Region
dwh.Dim_Benchmark	BenchmarkKey	BenchmarkCode	Type 1	6	Description, Currency

5.2 Dimension Table DDL Scripts

DIM-1: dwh.dim_advisor (SCD Type 1)

SCD Type 1 is applied to Dim_Advisor because changes to advisor region or experience years represent corrections rather than historical events. The current value always reflects the most accurate known state.

```
-- DIM 1: Advisor (SCD Type 1)
CREATE TABLE dwh.dim_advisor (
    advisor_sk          INT IDENTITY(1,1)    PRIMARY KEY,
    advisor_id          VARCHAR(20)          NOT NULL,    -- natural key
    advisor_name        VARCHAR(100)         NOT NULL,
    region              VARCHAR(50)          NOT NULL,
    join_date           DATE                  NOT NULL,
    experience_years     INT                  NOT NULL,
    -- Audit columns
    dwh_created_date    DATETIME              DEFAULT GETDATE(),
    dwh_updated_date    DATETIME              DEFAULT GETDATE(),
    is_active           CHAR(1)               DEFAULT 'Y'
);
```

DIM-2: dwh.dim_client (SCD Type 2)

SCD Type 2 is applied to Dim_Client because changes to a client's risk profile, segment, country, or assigned advisor are significant business events that must be preserved in history. Regulatory and compliance reporting may require knowing a client's profile at any past point in time.

```
-- DIM 2: Client (SCD Type 2)
CREATE TABLE dwh.dim_client (
    client_sk          INT IDENTITY(1,1)    PRIMARY KEY,
    client_id          VARCHAR(20)          NOT NULL,    -- natural key
    client_name        VARCHAR(100)         NOT NULL,
    segment            VARCHAR(50)          NOT NULL,
    risk_profile       VARCHAR(50)          NOT NULL,    -- NULL -> 'Unknown'
    primary_advisor_id VARCHAR(20)          NOT NULL,
    client_since       DATE                  NOT NULL,
    country            VARCHAR(50)          NOT NULL,
    -- SCD Type 2 versioning columns
    effective_start_date DATE                NOT NULL,
    effective_end_date  DATE                NULL,        -- NULL = current record
    is_current          CHAR(1)              NOT NULL DEFAULT 'Y',
    dwh_created_date    DATETIME              DEFAULT GETDATE(),
    dwh_updated_date    DATETIME              DEFAULT GETDATE()
);
```

DIM-3: dwh.dim_account (SCD Type 2)

SCD Type 2 is applied to Dim_Account because changes to account status (e.g., Active to Closed), account type, or base currency represent auditable business state transitions that must be preserved.

```
-- DIM 3: Account (SCD Type 2)
```

```

CREATE TABLE dwh.dim_account (
    account_sk          INT IDENTITY(1,1)    PRIMARY KEY,
    account_id          VARCHAR(20)          NOT NULL,    -- natural key
    client_id           VARCHAR(20)          NOT NULL,
    account_type        VARCHAR(50)          NOT NULL,
    open_date           DATE                  NOT NULL,
    base_currency       VARCHAR(10)          NOT NULL,
    status              VARCHAR(20)          NOT NULL,
    -- SCD Type 2 versioning columns
    effective_start_date DATE                  NOT NULL,
    effective_end_date  DATE                  NULL,
    is_current          CHAR(1)              NOT NULL DEFAULT 'Y',
    dwh_created_date    DATETIME              DEFAULT GETDATE(),
    dwh_updated_date    DATETIME              DEFAULT GETDATE()
);

```

DIM-4: dwh.dim_security (SCD Type 1)

SCD Type 1 is applied to Dim_Security. Changes to a security's sector, currency, or region represent classification corrections rather than meaningful business events. The Sector column handles 672 null values from Bond, Cash, and Commodity securities using the default value 'N/A'.

```

-- DIM 4: Security (SCD Type 1)
CREATE TABLE dwh.dim_security (
    security_sk          INT IDENTITY(1,1)    PRIMARY KEY,
    security_id          VARCHAR(20)          NOT NULL,    -- natural key
    ticker               VARCHAR(20)          NOT NULL,
    security_name        VARCHAR(100)         NOT NULL,
    asset_class          VARCHAR(50)          NOT NULL,
    sector               VARCHAR(50)          NOT NULL,    -- NULL -> 'N/A'
    currency             VARCHAR(10)          NOT NULL,
    region               VARCHAR(50)          NOT NULL,
    -- Audit columns
    dwh_created_date     DATETIME              DEFAULT GETDATE(),
    dwh_updated_date     DATETIME              DEFAULT GETDATE(),
    is_active            CHAR(1)              DEFAULT 'Y'
);

```

DIM-5: dwh.dim_benchmark (SCD Type 1)

The Benchmarks source file contains 4,020 rows representing daily index values for six benchmark codes. The dimension table stores only the six distinct benchmark codes with their descriptive metadata. The SP uses SELECT DISTINCT to deduplicate during loading.

```

-- DIM 5: Benchmark (SCD Type 1 - reference data)
CREATE TABLE dwh.dim_benchmark (
    benchmark_sk          INT IDENTITY(1,1)    PRIMARY KEY,
    benchmark_code        VARCHAR(20)          NOT NULL,    -- natural key
    description           VARCHAR(100)         NOT NULL,
    currency              VARCHAR(10)          NOT NULL,
    -- Audit columns

```

```

    dwh_created_date    DATETIME          DEFAULT GETDATE(),
    dwh_updated_date    DATETIME          DEFAULT GETDATE(),
    is_active           CHAR(1)           DEFAULT 'Y'
);

```

DIM-6: dwh.dim_date (Static Date Dimension)

A pre-populated date dimension table covering 2012-01-01 to 2026-12-31 is used across all fact tables. Date keys use the format YYYYMMDD as integers for efficient indexing. The table is populated once using a recursive CTE and does not require SCD logic since calendar dates are immutable.

```

-- DIM 6: Date (static - populated once)
CREATE TABLE dwh.dim_date (
    date_sk            INT                PRIMARY KEY, -- YYYYMMDD format
    full_date          DATE              NOT NULL,
    day_of_month       INT              NOT NULL,
    day_name           VARCHAR(10)       NOT NULL,
    week_number        INT              NOT NULL,
    month_number       INT              NOT NULL,
    month_name         VARCHAR(10)       NOT NULL,
    quarter_number     INT              NOT NULL,
    year_number        INT              NOT NULL,
    is_weekend         CHAR(1)          NOT NULL,
    is_month_end       CHAR(1)          NOT NULL
);

-- Populate dim_date (2012-01-01 to 2026-12-31)
WITH date_cte AS (
    SELECT CAST('2012-01-01' AS DATE) AS dt
    UNION ALL
    SELECT DATEADD(DAY, 1, dt)
    FROM date_cte WHERE dt < '2026-12-31'
)
INSERT INTO dwh.dim_date (
    date_sk, full_date, day_of_month, day_name,
    week_number, month_number, month_name,
    quarter_number, year_number, is_weekend, is_month_end
)
SELECT
    CONVERT(INT, FORMAT(dt, 'yyyyMMdd')), dt, DAY(dt),
    DATENAME(WEEKDAY, dt), DATEPART(WEEK, dt), MONTH(dt),
    DATENAME(MONTH, dt), DATEPART(QUARTER, dt), YEAR(dt),
    CASE WHEN DATEPART(WEEKDAY, dt) IN (1,7) THEN 'Y' ELSE 'N' END,
    CASE WHEN dt = EOMONTH(dt) THEN 'Y' ELSE 'N' END
FROM date_cte
OPTION (MAXRECURSION 10000);

```

5.3 Fact Tables

The data warehouse contains six fact tables, each recording measurable business events at an appropriate grain. Fact tables store surrogate key references to dimension tables

rather than natural keys, enabling efficient joins in Power BI. All fact tables are fully refreshed on every pipeline run using the TRUNCATE-and-reload pattern.

Natural key columns are retained alongside surrogate keys in each fact table to facilitate data lineage tracing and debugging — allowing engineers to trace any DWH record back to its source file row without requiring a join back through the dimension tables.

Fact Table	Grain	Dimension FKs	Row Count	Key Measures
dwh.Fact_Holdings	Account x Security x Date	AccountKey, SecurityKey, DateKey	96,027	Quantity, MarketValue
dwh.Fact_Trades	One row per trade	AccountKey, SecurityKey, TradeDateKey, SettleDateKey	100,000	Price, Quantity, Amount, SettleLagDays
dwh.Fact_Cashflows	One row per cashflow	AccountKey, DateKey	30,000	Amount, IsInflow (Y/N)
dwh.Fact_Fees	One row per fee charge	AccountKey, DateKey	18,000	Amount
dwh.Fact_Performance	Account x Month	AccountKey, BenchmarkKey, DateKey	110,000	ReturnPct, BenchmarkReturnPct, ExcessReturnPct, CumulativeReturnPct
dwh.Fact_Security_Price	Security x Date	SecurityKey, DateKey	301,022	ClosePrice (478 NULLs allowed)

FACT-1: dwh.fact_holding

```
-- FACT 1: Holdings (AUM source - point-in-time snapshot)
CREATE TABLE dwh.fact_holding (
    holding_sk          INT IDENTITY(1,1)    PRIMARY KEY,
    account_sk          INT                  NOT NULL REFERENCES
dwh.dim_account(account_sk),
    security_sk         INT                  NOT NULL REFERENCES
dwh.dim_security(security_sk),
    date_sk             INT                  NOT NULL REFERENCES
dwh.dim_date(date_sk),
    -- Natural keys for traceability
    account_id          VARCHAR(20)         NOT NULL,
    security_id          VARCHAR(20)         NOT NULL,
    valuation_date      DATE                 NOT NULL,
    -- Measures
    quantity            DECIMAL(18,4)        NOT NULL,
    market_value        DECIMAL(18,2)        NOT NULL,
    dwh_load_timestamp  DATETIME             DEFAULT GETDATE()
);
```


FACT-2: dwh.fact_trade

The Fact_Trade table uses two date foreign keys: trade_date_sk and settle_date_sk — both referencing dwh.dim_date), enabling settlement efficiency analysis in Power BI.

```
-- FACT 2: Trades (transactional)
CREATE TABLE dwh.fact_trade (
    trade_sk          INT IDENTITY(1,1)    PRIMARY KEY,
    account_sk        INT                  NOT NULL REFERENCES
dwh.dim_account(account_sk),
    security_sk       INT                  NOT NULL REFERENCES
dwh.dim_security(security_sk),
    trade_date_sk     INT                  NOT NULL REFERENCES
dwh.dim_date(date_sk),
    settle_date_sk    INT                  NOT NULL REFERENCES
dwh.dim_date(date_sk),
    trade_id          VARCHAR(20)          NOT NULL,
    account_id        VARCHAR(20)          NOT NULL,
    security_id       VARCHAR(20)          NOT NULL,
    trade_date        DATE                  NOT NULL,
    settle_date       DATE                  NOT NULL,
    side              VARCHAR(10)          NOT NULL,
    price             DECIMAL(18,4)        NOT NULL,
    quantity          DECIMAL(18,4)        NOT NULL,
    amount            DECIMAL(18,2)        NOT NULL,
    settle_lag_days   INT                  NOT NULL,    -- derived:
DATEDIFF(TradeDate, SettleDate)
    dwh_load_timestamp DATETIME            DEFAULT GETDATE()
);
```

FACT-3: dwh.fact_cashflow

The is_inflow column is a derived flag (Y/N) indicating whether the cashflow represents money coming into the account (positive Amount) or going out (negative Amount). This simplifies Power BI filtering for inflow vs. outflow analysis.

```
-- FACT 3: Cashflows (transactional)
CREATE TABLE dwh.fact_cashflow (
    cashflow_sk       INT IDENTITY(1,1)    PRIMARY KEY,
    account_sk        INT                  NOT NULL REFERENCES
dwh.dim_account(account_sk),
    date_sk           INT                  NOT NULL REFERENCES
dwh.dim_date(date_sk),
    cashflow_id       VARCHAR(20)          NOT NULL,
    account_id        VARCHAR(20)          NOT NULL,
    cashflow_date     DATE                  NOT NULL,
    cashflow_type     VARCHAR(30)          NOT NULL,
    amount            DECIMAL(18,2)        NOT NULL,
    is_inflow         CHAR(1)              NOT NULL,    -- Y/N derived from sign of
Amount
    dwh_load_timestamp DATETIME            DEFAULT GETDATE()
);
```

FACT-4: dwh.fact_fee

```
-- FACT 4: Fees (transactional)
CREATE TABLE dwh.fact_fee (
    fee_sk          INT IDENTITY(1,1)    PRIMARY KEY,
    account_sk      INT                  NOT NULL REFERENCES
dwh.dim_account(account_sk),
    date_sk         INT                  NOT NULL REFERENCES
dwh.dim_date(date_sk),
    fee_id          VARCHAR(20)          NOT NULL,
    account_id      VARCHAR(20)          NOT NULL,
    fee_date        DATE                  NOT NULL,
    fee_type        VARCHAR(50)          NOT NULL,
    amount          DECIMAL(18,2)        NOT NULL,
    dwh_load_timestamp DATETIME          DEFAULT GETDATE()
);
```

FACT-5: dwh.fact_security_price

ClosePrice is allowed to be NULL in this table. The 478 source records with missing ClosePrice are loaded with NULL values rather than being discarded, and are flagged in the audit log. This ensures that the date-security record exists in the fact table even when the price feed failed to deliver a value.

```
-- FACT 5: Security Prices (daily)
CREATE TABLE dwh.fact_security_price (
    price_sk        INT IDENTITY(1,1)    PRIMARY KEY,
    security_sk     INT                  NOT NULL REFERENCES
dwh.dim_security(security_sk),
    date_sk         INT                  NOT NULL REFERENCES
dwh.dim_date(date_sk),
    security_id     VARCHAR(20)          NOT NULL,
    price_date      DATE                  NOT NULL,
    close_price     DECIMAL(18,4)        NULL,      -- NULL allowed; 478
missing prices
    dwh_load_timestamp DATETIME          DEFAULT GETDATE()
);
```

FACT-6: dwh.fact_performance

ReturnPct is allowed to be NULL (109 records). These represent months where the account's return could not be calculated from the source system. BenchmarkReturnPct, ExcessReturnPct, and CumulativeReturnPct remain populated and usable for benchmark comparison even when ReturnPct is missing.

```
-- FACT 6: Performance (monthly)
CREATE TABLE dwh.fact_performance (
    performance_sk  INT IDENTITY(1,1)    PRIMARY KEY,
```

```

        account_sk          INT          NOT NULL REFERENCES
dwh.dim_account(account_sk),
        benchmark_sk       INT          NOT NULL REFERENCES
dwh.dim_benchmark(benchmark_sk),
        date_sk            INT          NOT NULL REFERENCES
dwh.dim_date(date_sk),
        account_id         VARCHAR(20)   NOT NULL,
        benchmark_code     VARCHAR(20)   NOT NULL,
        month_start        DATE          NOT NULL,
        return_pct         DECIMAL(10,6)  NULL,      -- NULL allowed (109
records)
        benchmark_return_pct DECIMAL(10,6) NOT NULL,
        excess_return_pct  DECIMAL(10,6) NOT NULL,
        cumulative_return_pct DECIMAL(10,6) NOT NULL,
        dwh_load_timestamp DATETIME      DEFAULT GETDATE()
);

```

5.4 Audit Table

The `audit_pipeline_log` table records the execution status of each pipeline run. Every pipeline writes an entry with the pipeline name, source and target table names, start and end times, row counts (read, loaded, and rejected), and a status field (RUNNING, SUCCESS, or FAILED). This table provides operational monitoring capability and supports troubleshooting and compliance reporting.

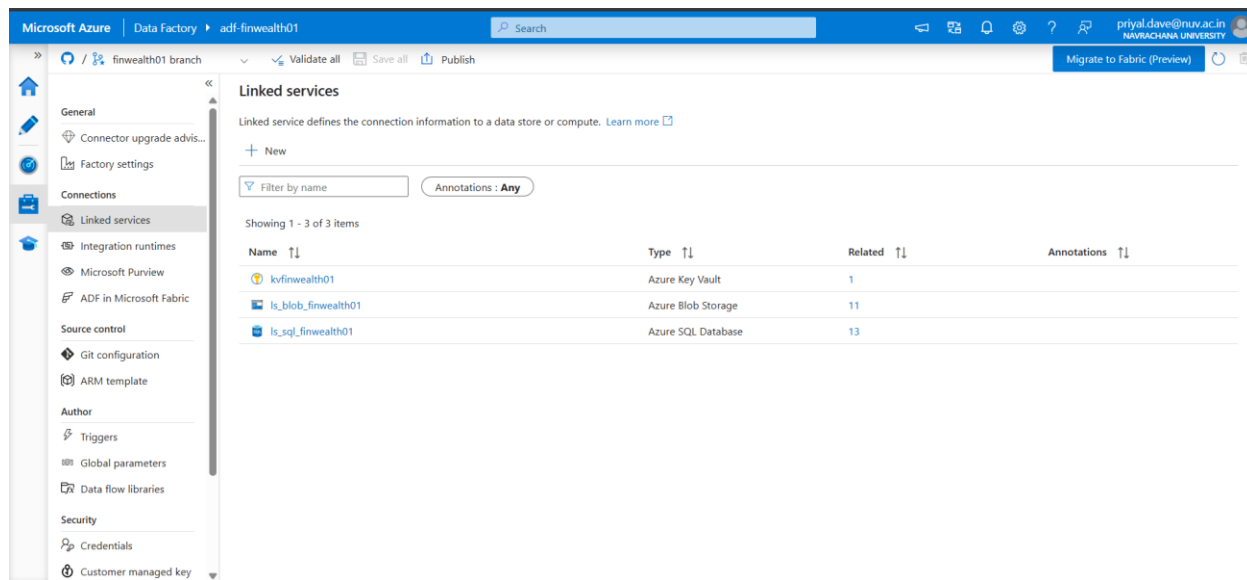
```

CREATE TABLE dbo.audit_pipeline_log (
    log_id          INT IDENTITY(1,1) PRIMARY KEY,
    pipeline_name   VARCHAR(100)      NOT NULL,
    source_file     VARCHAR(100)      NOT NULL,
    target_table    VARCHAR(100)      NOT NULL,
    run_start_time  DATETIME          NOT NULL,
    run_end_time    DATETIME          NULL,
    rows_read       INT              NULL,
    rows_loaded     INT              NULL,
    rows_rejected   INT              NULL,
    status          VARCHAR(20)       NOT NULL, -- RUNNING | SUCCESS |
FAILED
    error_message   VARCHAR(2000)     NULL,
    loaded_by       VARCHAR(100)      DEFAULT 'ADF'
);

```

6. Azure Data Factory Pipeline — Configuration and Design

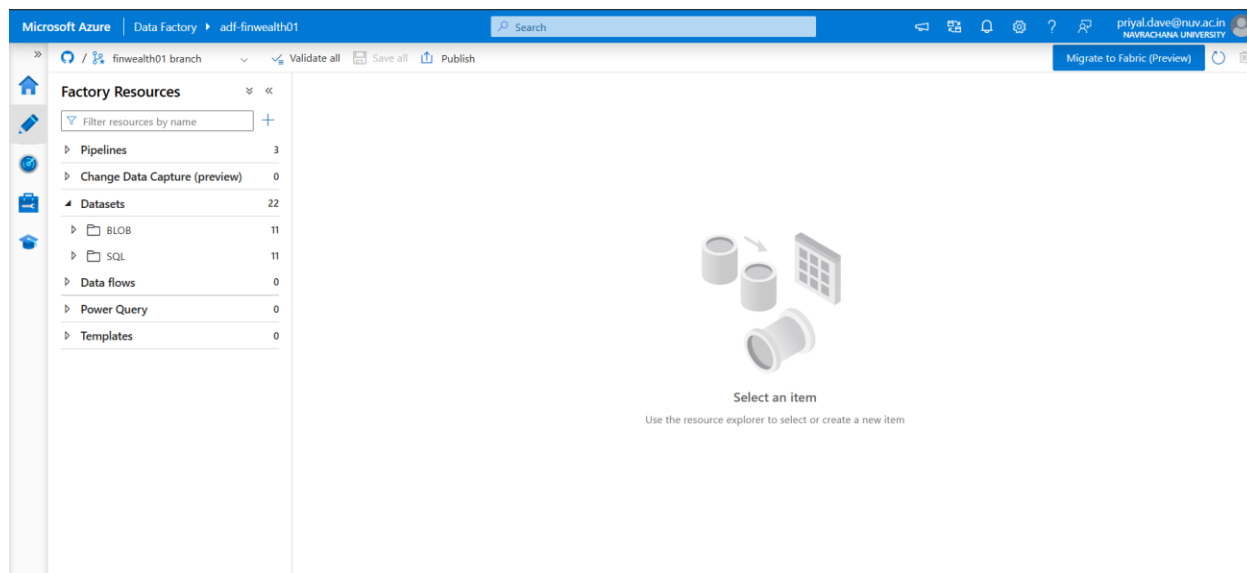
6.1 Linked Services



Three linked services were created in ADF to establish authenticated connections to the required Azure resources. The Key Vault linked service provides secure credential retrieval, eliminating the need to embed sensitive connection strings directly in linked service definitions.

Linked Service Name	Type	Connects To	Purpose
ls_blob_finwealth01	Azure Blob Storage	stfinwealth01 / raw-data container	Read source CSV files for Copy Activities
ls_sql_finwealth01	Azure SQL Database	sql-finwealth01-server / Sql-db-finwealth01	Write to staging tables and execute stored procedures
kvfinwealth01	Azure Key Vault	kv-finwealth01	Securely retrieve connection strings for other linked services

6.2 Datasets



Twenty-two datasets were created. 11 DelimitedText datasets pointing to CSV files in Azure Blob Storage, and eleven Azure SQL Database datasets pointing to the corresponding staging tables. All CSV datasets have 'First row as header' enabled and schemas imported from the connection.

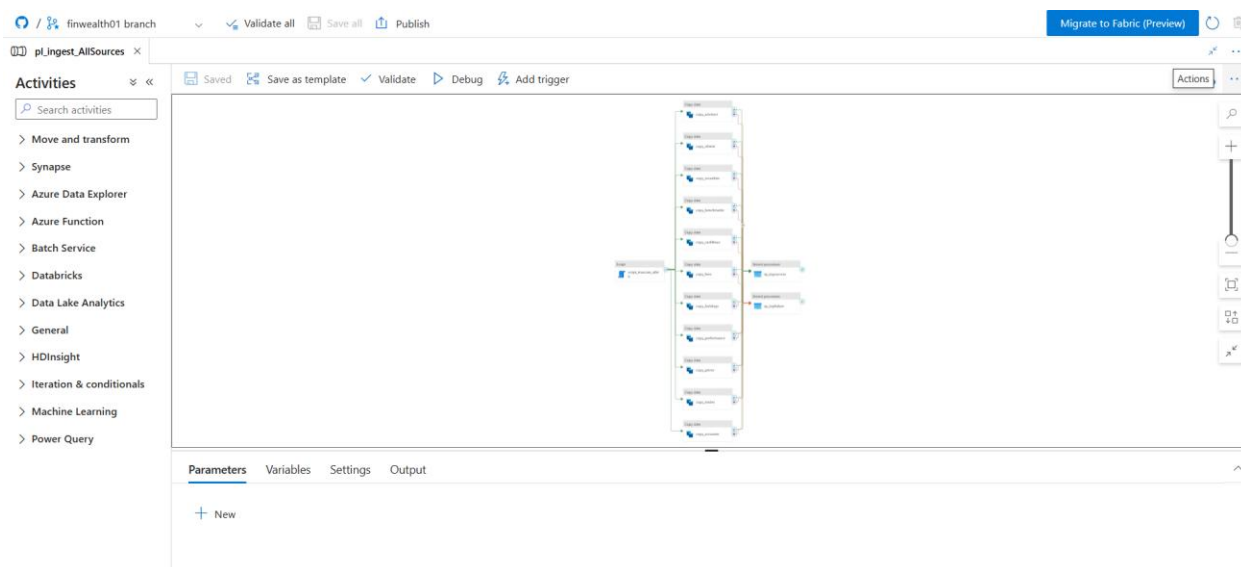
BLOB Source Datasets (ds_blob folder)

Dataset Name	Source File	Linked Service
ds_blob_advisors	Advisors.csv	ls_blob_finwealth01
ds_blob_clients	Clients.csv	ls_blob_finwealth01
ds_blob_accounts	Accounts.csv	ls_blob_finwealth01
ds_blob_securities	Securities.csv	ls_blob_finwealth01
ds_blob_benchmarks	Benchmarks.csv	ls_blob_finwealth01
ds_blob_cashflows	Cashflows.csv	ls_blob_finwealth01
ds_blob_fees	Fees.csv	ls_blob_finwealth01
ds_blob_holdings	Holdings.csv	ls_blob_finwealth01
ds_blob_performance	Performance.csv	ls_blob_finwealth01
ds_blob_prices	Prices.csv	ls_blob_finwealth01
ds_blob_trades	Trades.csv	ls_blob_finwealth01

SQL Staging Datasets (ds_sql folder)

Dataset Name	Target Table	Linked Service
ds_sql_stg_advisors	stg.stg_advisors	ls_sql_finwealth01
ds_sql_stg_clients	stg.stg_clients	ls_sql_finwealth01
ds_sql_stg_accounts	stg.stg_accounts	ls_sql_finwealth01
ds_sql_stg_securities	stg.stg_securities	ls_sql_finwealth01
ds_sql_stg_benchmarks	stg.stg_benchmarks	ls_sql_finwealth01
ds_sql_stg_cashflows	stg.stg_cashflows	ls_sql_finwealth01
ds_sql_stg_fees	stg.stg_fees	ls_sql_finwealth01
ds_sql_stg_holdings	stg.stg_holdings	ls_sql_finwealth01
ds_sql_stg_performance	stg.stg_performance	ls_sql_finwealth01
ds_sql_stg_prices	stg.stg_prices	ls_sql_finwealth01
ds_sql_stg_trades	stg.stg_trades	ls_sql_finwealth01

6.3 Pipeline 1 — pl_ingest_AllSources



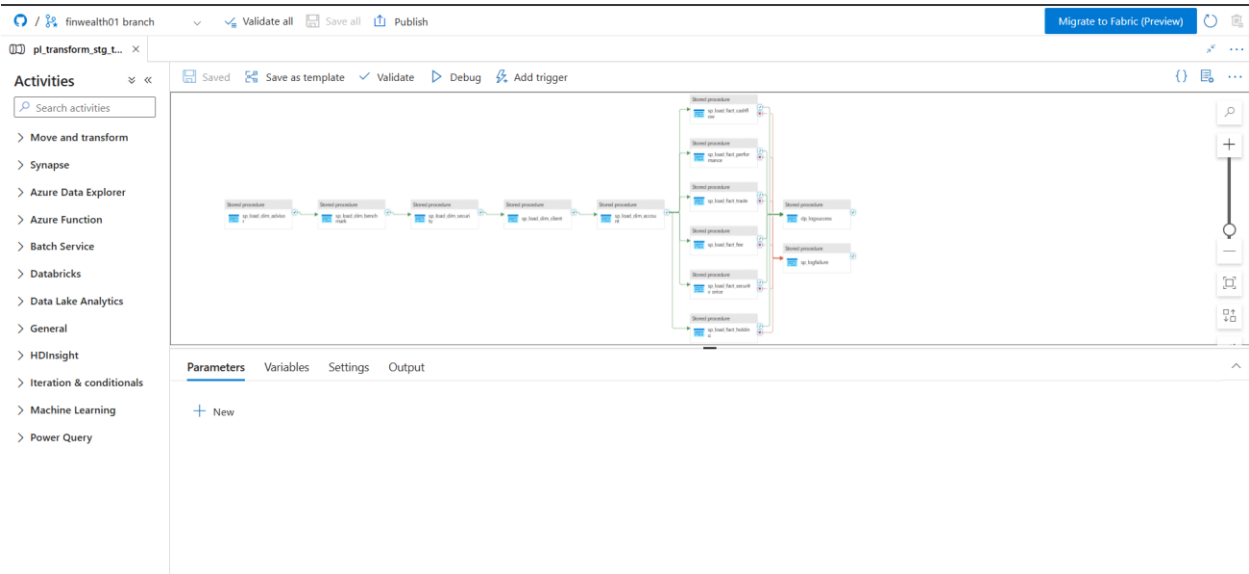
This pipeline is responsible for loading all eleven source CSV files from Azure Blob Storage into the corresponding SQL staging tables. It contains eleven Copy Activities configured to run in parallel — with no dependency relationships between them — significantly reducing total ingestion time compared to sequential execution.

Each Copy Activity is configured with a pre-copy script that executes a TRUNCATE TABLE statement on the target staging table before loading, ensuring a complete and clean full

reload on every run. The pipeline is triggered by the master pipeline and completes in approximately four to five minutes for all eleven files combined.

Copy Activity	Source Dataset	Sink Dataset	Pre-Copy Script	Approx. Duration
Copy_Advisors	ds_blob_advisors	ds_sql_stg_advisors	TRUNCATE TABLE stg.stg_Advisors	~25 sec
Copy_Clients	ds_blob_clients	ds_sql_stg_clients	TRUNCATE TABLE stg.stg_Clients	~34 sec
Copy_Accounts	ds_blob_accounts	ds_sql_stg_accounts	TRUNCATE TABLE stg.stg_Accounts	~1 min 3 sec
Copy_Securities	ds_blob_securities	ds_sql_stg_securities	TRUNCATE TABLE stg.stg_Securities	~36 sec
Copy_Benchmarks	ds_blob_benchmarks	ds_sql_stg_benchmarks	TRUNCATE TABLE stg.stg_Benchmarks	~40 sec
Copy_Holdings	ds_blob_holdings	ds_sql_stg_holdings	TRUNCATE TABLE stg.stg_Holdings	~1 min 35 sec
Copy_Trades	ds_blob_trades	ds_sql_stg_trades	TRUNCATE TABLE stg.stg_Trades	~2 min 53 sec
Copy_Cashflows	ds_blob_cashflows	ds_sql_stg_cashflows	TRUNCATE TABLE stg.stg_Cashflows	~1 min 38 sec
Copy_Fees	ds_blob_fees	ds_sql_stg_fees	TRUNCATE TABLE stg.stg_Fees	~45 sec
Copy_Performance	ds_blob_performance	ds_sql_stg_performance	TRUNCATE TABLE stg.stg_Performance	~2 min 41 sec
Copy_Prices	ds_blob_prices	ds_sql_stg_prices	TRUNCATE TABLE stg.stg_Prices	~3 min 34 sec

6.4 Pipeline 2 — pl_transform_stg_to_dwh



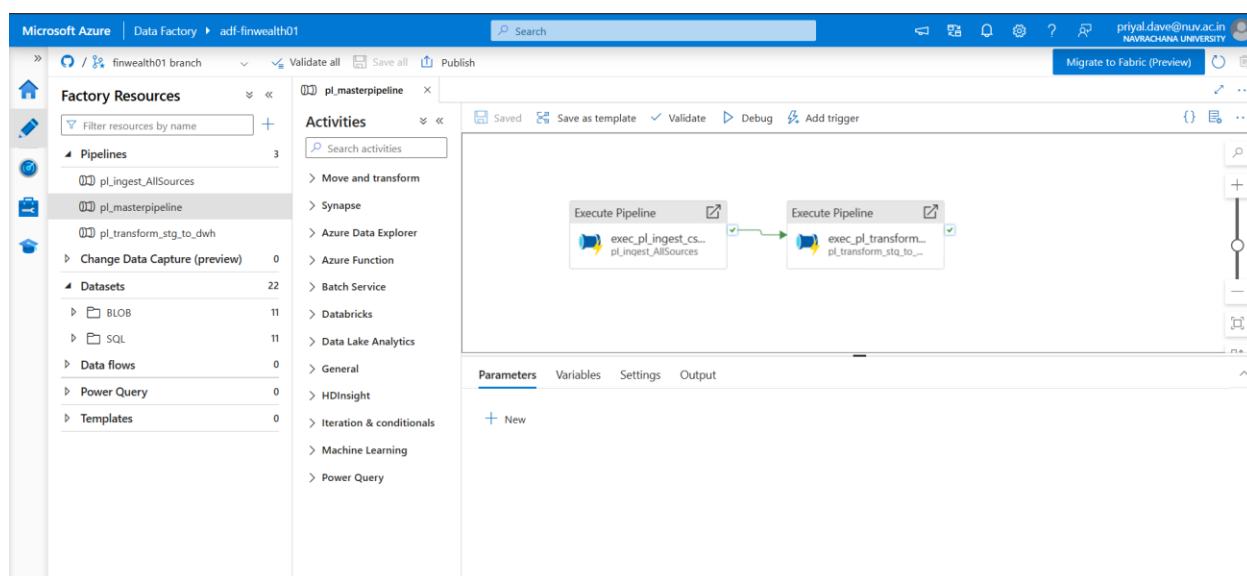
This pipeline is responsible for executing all eleven stored procedures that transform and load data from the staging layer into the data warehouse. It uses Stored Procedure Activities connected to the `ls_sql_finwealth_dwh` linked service. Execution order is critical: all dimension stored procedures must complete successfully before any fact stored procedure begins, because fact procedures depend on the surrogate keys generated during dimension loading.

The five dimension SPs are chained sequentially using success dependencies. Once all dimension SPs complete, the six fact SPs run in parallel for maximum throughput. A final audit logging SP executes after all fact loads to record the transform pipeline's summary statistics.

Order	SP Activity Name	Stored Procedure	Type / Pattern
1	Load Dim Advisor	dwh.usp_load_dim_advisor	SCD Type 1
2	Load Dim Benchmark	dwh.usp_load_dim_benchmark	SCD Type 1
3	Load Dim Security	dwh.usp_load_dim_security	SCD Type 1
4	Load Dim Client	dwh.usp_load_dim_client	SCD Type 2
5	Load Dim Account	dwh.usp_load_dim_account	SCD Type 2
6 (parallel)	Load Fact Holdings	dwh.usp_load_fact_holding	Full Refresh
6 (parallel)	Load Fact Trades	dwh.usp_load_fact_trade	Full Refresh + Dedup
6 (parallel)	Load Fact Cashflows	dwh.usp_load_fact_cashflow	Full Refresh
6 (parallel)	Load Fact Fees	dwh.usp_load_fact_fee	Full Refresh

Order	SP Activity Name	Stored Procedure	Type / Pattern
6 (parallel)	Load Fact Security Price	dwh.usp_load_fact_security_price	Full Refresh
6 (parallel)	Load Fact Performance	dwh.usp_load_fact_performance	Full Refresh
7	Log Transform Summary	dbo.usp_log_transform_summary	Audit Logging

6.5 Pipeline 3 — pl_masterpipeline (Orchestrator)



The master pipeline serves as the single entry point for executing the complete end-to-end data pipeline. It contains two Execute Pipeline activities connected by a success dependency. Only this pipeline should be triggered manually or via a scheduled trigger. This design ensures that the ingestion and transformation stages always execute in the correct sequence and that a failure in Stage 1 prevents Stage 2 from running with incomplete data.

Activity Name	Type	Invoked Pipeline	Wait on Completion
exec_pl_ingest_AllSources	Execute Pipeline	pl_ingest_AllSources	Yes — blocks until all 11 Copy Activities succeed
exec_pl_transform_stg_to_dwh	Execute Pipeline	pl_transform_stg_to_dwh	Yes — blocks until all 11 SP Activities succeed

7. Transformation Logic — Stored Procedures

7.1 Stored Procedures Summary

SP Name	Schema	Purpose	Key Logic
usp_load_dim_advisor	dwh	Load Dim_Advisor	SCD1: UPDATE changed records, INSERT new. Region NULL/blank → 'Unknown'
usp_load_dim_benchmark	dwh	Load Dim_Benchmark	SCD1: SELECT DISTINCT BenchmarkCode. Collapses 4,020 rows → 6 rows
usp_load_dim_security	dwh	Load Dim_Security	SCD1: UPDATE/INSERT. Sector NULL/blank → 'N/A'
usp_load_dim_client	dwh	Load Dim_Client	SCD2: EXPIRE current record, INSERT new version. RiskProfile NULL → 'Unknown'
usp_load_dim_account	dwh	Load Dim_Account	SCD2: EXPIRE current record, INSERT new version
usp_load_fact_holding	dwh	Load Fact_Holdings	TRUNCATE + INSERT. INNER JOIN 3 dims. Resolves surrogate keys
usp_load_fact_trade	dwh	Load Fact_Trades	TRUNCATE + INSERT with dedup/deamend logic. SettleLagDays derived
usp_load_fact_cashflow	dwh	Load Fact_Cashflows	TRUNCATE + INSERT. is_inflow flag derived from Amount sign
usp_load_fact_fee	dwh	Load Fact_Fees	TRUNCATE + INSERT. Joins dim_account and dim_date
usp_load_fact_security_price	dwh	Load Fact_Security_Price	TRUNCATE + INSERT. ClosePrice blank/NULL → explicit NULL
usp_load_fact_performance	dwh	Load Fact_Performance	TRUNCATE + INSERT. ReturnPct blank/NULL → NULL. Joins 3 dims
usp_log_ingest_summary	dbo	Audit ingest pipeline	Writes pipeline run status and row counts to audit_pipeline_log
usp_log_transform_summary	dbo	Audit transform pipeline	Writes transform pipeline run status to audit_pipeline_log

7.2 Dimension Stored Procedures

SP-1: dwh.usp_load_dim_advisor (SCD Type 1)

This procedure implements SCD Type 1 logic for the Advisor dimension. It first updates any existing advisor records where the name, region, or experience years have changed, then inserts any new advisor IDs that do not yet exist in the dimension table. The `ISNULL(NULLIF(src.Region, ''), 'Unknown')` expression handles both NULL values and empty strings from the source.

```
ALTER PROCEDURE dwh.usp_load_dim_advisor
AS
BEGIN
    SET NOCOUNT ON;

    -- Step 1: UPDATE existing records where attributes have changed (SCD Type 1
    overwrite)
    UPDATE dwh.dim_advisor
    SET
        advisor_name      = src.AdvisorName,
        region             = ISNULL(NULLIF(src.Region, ''), 'Unknown'),
        join_date          = CAST(src.JoinDate AS DATE),
        experience_years    = CAST(src.ExperienceYears AS INT),
        dwh_updated_date   = GETDATE()
    FROM dwh.dim_advisor tgt
    INNER JOIN stg.stg_advisors src
        ON tgt.advisor_id = src.AdvisorID
    WHERE
        tgt.advisor_name    != src.AdvisorName OR
        tgt.region          != ISNULL(NULLIF(src.Region, ''), 'Unknown') OR
        tgt.experience_years != CAST(src.ExperienceYears AS INT);

    -- Step 2: INSERT new advisors that do not exist yet
    INSERT INTO dwh.dim_advisor (
        advisor_id, advisor_name, region,
        join_date, experience_years, dwh_created_date, dwh_updated_date, is_active
    )
    SELECT
        src.AdvisorID, src.AdvisorName,
        ISNULL(NULLIF(src.Region, ''), 'Unknown'),
        CAST(src.JoinDate AS DATE), CAST(src.ExperienceYears AS INT),
        GETDATE(), GETDATE(), 'Y'
    FROM stg.stg_advisors src
    WHERE NOT EXISTS (
        SELECT 1 FROM dwh.dim_advisor tgt
        WHERE tgt.advisor_id = src.AdvisorID
    );
END;
```

SP-2: dwh.usp_load_dim_benchmark (SCD Type 1)

This procedure uses `SELECT DISTINCT` on BenchmarkCode to collapse the 4,020 daily index value rows in the staging table into the 6 unique benchmark code records required by the dimension table. It then applies standard SCD Type 1 update and insert logic.

```

CREATE PROCEDURE dwh.usp_load_dim_benchmark
AS
BEGIN
    SET NOCOUNT ON;

    -- UPDATE existing records where description or currency changed
    UPDATE dwh.dim_benchmark
    SET
        description      = src.Description,
        currency          = src.Currency,
        dwh_updated_date = GETDATE()
    FROM dwh.dim_benchmark tgt
    INNER JOIN stg.stg_benchmarks src
        ON tgt.benchmark_code = src.BenchmarkCode
    WHERE tgt.description != src.Description OR tgt.currency != src.Currency;

    -- INSERT new benchmark codes (SELECT DISTINCT collapses 4020 rows -> 6)
    INSERT INTO dwh.dim_benchmark (
        benchmark_code, description, currency,
        dwh_created_date, dwh_updated_date, is_active
    )
    SELECT DISTINCT
        src.BenchmarkCode, src.Description, src.Currency,
        GETDATE(), GETDATE(), 'Y'
    FROM stg.stg_benchmarks src
    WHERE NOT EXISTS (
        SELECT 1 FROM dwh.dim_benchmark tgt
        WHERE tgt.benchmark_code = src.BenchmarkCode
    );
END;

```

SP-3: dwh.usp_load_dim_security (SCD Type 1)

This procedure handles the 672 null Sector values using ISNULL(NULLIF(src.Sector, ''), 'N/A'). This is correct business logic: Bond, Cash, and Commodity asset classes genuinely have no sector classification.

```

CREATE PROCEDURE dwh.usp_load_dim_security
AS
BEGIN
    SET NOCOUNT ON;

    -- UPDATE existing records (SCD Type 1 overwrite)
    UPDATE dwh.dim_security
    SET
        ticker          = src.Ticker,
        security_name    = src.SecurityName,
        asset_class      = src.AssetClass,
        sector           = ISNULL(NULLIF(src.Sector, ''), 'N/A'),
        currency         = src.Currency,
        region           = src.Region,
        dwh_updated_date = GETDATE()
    FROM dwh.dim_security tgt
    INNER JOIN stg.stg_securities src

```

```

        ON tgt.security_id = src.SecurityID
WHERE
    tgt.ticker      != src.Ticker OR
    tgt.asset_class != src.AssetClass OR
    tgt.sector      != ISNULL(NULLIF(src.Sector, ''), 'N/A') OR
    tgt.currency    != src.Currency OR
    tgt.region      != src.Region;

-- INSERT new securities
INSERT INTO dwh.dim_security (
    security_id, ticker, security_name, asset_class,
    sector, currency, region, dwh_created_date, dwh_updated_date, is_active
)
SELECT
    src.SecurityID, src.Ticker, src.SecurityName, src.AssetClass,
    ISNULL(NULLIF(src.Sector, ''), 'N/A'),
    src.Currency, src.Region, GETDATE(), GETDATE(), 'Y'
FROM stg.stg_securities src
WHERE NOT EXISTS (
    SELECT 1 FROM dwh.dim_security tgt
    WHERE tgt.security_id = src.SecurityID
);
END;

```

SP-4: dwh.usp_load_dim_client (SCD Type 2)

This procedure implements full SCD Type 2 logic. When a change is detected in any tracked attribute (Segment, RiskProfile, Country, or PrimaryAdvisorID), the current active record is expired by setting `is_current = 'N'` and `effective_end_date` to today's date. A new version of the record is then inserted with `is_current = 'Y'` and `effective_start_date` set to today. Brand-new clients are inserted directly as current records.

```

CREATE PROCEDURE dwh.usp_load_dim_client
AS
BEGIN
    SET NOCOUNT ON;

    -- Step 1: EXPIRE changed records (SCD Type 2 versioning)
    UPDATE dwh.dim_client
    SET
        effective_end_date = CAST(GETDATE() AS DATE),
        is_current         = 'N',
        dwh_updated_date   = GETDATE()
    FROM dwh.dim_client tgt
    INNER JOIN stg.stg_clients src
        ON tgt.client_id = src.ClientID AND tgt.is_current = 'Y'
    WHERE
        tgt.segment      != src.Segment OR
        tgt.risk_profile  != ISNULL(NULLIF(src.RiskProfile, ''), 'Unknown')
OR
        tgt.country      != src.Country OR
        tgt.primary_advisor_id != src.PrimaryAdvisorID;

    -- Step 2: INSERT new version for changed clients
    INSERT INTO dwh.dim_client (

```

```

        client_id, client_name, segment, risk_profile, primary_advisor_id,
        client_since, country, effective_start_date, effective_end_date,
        is_current, dwh_created_date, dwh_updated_date
    )
    SELECT
        src.ClientID, src.ClientName, src.Segment,
        ISNULL(NULLIF(src.RiskProfile, ''), 'Unknown'),
        src.PrimaryAdvisorID, CAST(src.ClientSince AS DATE), src.Country,
        CAST(GETDATE() AS DATE), NULL, 'Y', GETDATE(), GETDATE()
    FROM stg.stg_clients src
    INNER JOIN dwh.dim_client tgt
        ON tgt.client_id = src.ClientID
        AND tgt.is_current = 'N'
        AND tgt.effective_end_date = CAST(GETDATE() AS DATE)
    WHERE NOT EXISTS (
        SELECT 1 FROM dwh.dim_client cur
        WHERE cur.client_id = src.ClientID AND cur.is_current = 'Y'
    );

-- Step 3: INSERT brand new clients (first-time load)
INSERT INTO dwh.dim_client (
    client_id, client_name, segment, risk_profile, primary_advisor_id,
    client_since, country, effective_start_date, effective_end_date,
    is_current, dwh_created_date, dwh_updated_date
)
SELECT
    src.ClientID, src.ClientName, src.Segment,
    ISNULL(NULLIF(src.RiskProfile, ''), 'Unknown'),
    src.PrimaryAdvisorID, CAST(src.ClientSince AS DATE), src.Country,
    CAST(GETDATE() AS DATE), NULL, 'Y', GETDATE(), GETDATE()
FROM stg.stg_clients src
WHERE NOT EXISTS (
    SELECT 1 FROM dwh.dim_client tgt WHERE tgt.client_id = src.ClientID
);
END;
```

SP-5: dwh.usp_load_dim_account (SCD Type 2)

This procedure applies the same SCD Type 2 pattern as Dim_Client, tracking changes to account Status, AccountType, and BaseCurrency — attributes that have auditable significance in a regulated wealth management context.

```

CREATE PROCEDURE dwh.usp_load_dim_account
AS
BEGIN
    SET NOCOUNT ON;

    -- Step 1: EXPIRE changed records
    UPDATE dwh.dim_account
    SET
        effective_end_date = CAST(GETDATE() AS DATE),
        is_current         = 'N',
        dwh_updated_date   = GETDATE()
    FROM dwh.dim_account tgt
    INNER JOIN stg.stg_accounts src
        ON tgt.account_id = src.AccountID AND tgt.is_current = 'Y'
```

```

WHERE
    tgt.status          != src.Status OR
    tgt.account_type    != src.AccountType OR
    tgt.base_currency   != src.BaseCurrency;

-- Step 2: INSERT new version for changed accounts
INSERT INTO dwh.dim_account (
    account_id, client_id, account_type, open_date, base_currency, status,
    effective_start_date, effective_end_date, is_current,
    dwh_created_date, dwh_updated_date
)
SELECT
    src.AccountID, src.ClientID, src.AccountType,
    CAST(src.OpenDate AS DATE), src.BaseCurrency, src.Status,
    CAST(GETDATE() AS DATE), NULL, 'Y', GETDATE(), GETDATE()
FROM stg.stg_accounts src
INNER JOIN dwh.dim_account tgt
    ON tgt.account_id = src.AccountID AND tgt.is_current = 'N'
    AND tgt.effective_end_date = CAST(GETDATE() AS DATE)
WHERE NOT EXISTS (
    SELECT 1 FROM dwh.dim_account cur
    WHERE cur.account_id = src.AccountID AND cur.is_current = 'Y'
);

-- Step 3: INSERT new accounts
INSERT INTO dwh.dim_account (
    account_id, client_id, account_type, open_date, base_currency, status,
    effective_start_date, effective_end_date, is_current,
    dwh_created_date, dwh_updated_date
)
SELECT
    src.AccountID, src.ClientID, src.AccountType,
    CAST(src.OpenDate AS DATE), src.BaseCurrency, src.Status,
    CAST(GETDATE() AS DATE), NULL, 'Y', GETDATE(), GETDATE()
FROM stg.stg_accounts src
WHERE NOT EXISTS (
    SELECT 1 FROM dwh.dim_account tgt WHERE tgt.account_id = src.AccountID
);
END;

```

7.3 Fact Stored Procedures

SP-6: dwh.usp_load_fact_holding

All fact procedures follow the TRUNCATE-and-full-reload pattern. Surrogate keys are resolved by performing INNER JOINS against the dimension tables. Only records where all required dimension keys can be resolved are inserted into the fact table — ensuring referential integrity without the overhead of separate validation queries.

```

CREATE PROCEDURE dwh.usp_load_fact_holding
AS
BEGIN
    SET NOCOUNT ON;

```

```

TRUNCATE TABLE dwh.fact_holding;

INSERT INTO dwh.fact_holding (
    account_sk, security_sk, date_sk,
    account_id, security_id, valuation_date, quantity, market_value
)
SELECT
    da.account_sk, ds.security_sk, dd.date_sk,
    src.AccountID, src.SecurityID,
    CAST(src.ValuationDate AS DATE),
    CAST(src.Quantity AS DECIMAL(18,4)),
    CAST(src.MarketValue AS DECIMAL(18,2))
FROM stg.stg_holdings src
INNER JOIN dwh.dim_account da
    ON da.account_id = src.AccountID AND da.is_current = 'Y'
INNER JOIN dwh.dim_security ds
    ON ds.security_id = src.SecurityID AND ds.is_active = 'Y'
INNER JOIN dwh.dim_date dd
    ON dd.full_date = CAST(src.ValuationDate AS DATE);
END;

```

SP-7: dwh.usp_load_fact_trade (with Deduplication Logic)

The Trades staging table contained 400 exact duplicate rows and 3 records with amended trade amounts (same TradeID, different Amount values). A two-step deduplication approach was implemented:

- Step 1 — Remove exact duplicates using SELECT DISTINCT across all columns.
- Step 2 — For same TradeID with differing amounts (amended trades), select the row where Amount most closely matches Price x Quantity using ROW_NUMBER() with an ABS() ordering expression.

```

ALTER PROCEDURE dwh.usp_load_fact_trade
AS
BEGIN
    SET NOCOUNT ON;
    TRUNCATE TABLE dwh.fact_trade;

    INSERT INTO dwh.fact_trade (
        account_sk, security_sk, trade_date_sk, settle_date_sk,
        trade_id, account_id, security_id, trade_date, settle_date,
        side, price, quantity, amount, settle_lag_days
    )
    SELECT
        da.account_sk, ds.security_sk,
        dd_trade.date_sk, dd_settle.date_sk,
        src.TradeID, src.AccountID, src.SecurityID,
        CAST(src.TradeDate AS DATE), CAST(src.SettleDate AS DATE),
        src.Side,
        CAST(src.Price AS DECIMAL(18,4)),
        CAST(src.Quantity AS DECIMAL(18,4)),
        CAST(src.Amount AS DECIMAL(18,2)),
        DATEDIFF(DAY, CAST(src.TradeDate AS DATE), CAST(src.SettleDate AS DATE))
    FROM (

```



```

-- Final CTE: one row per TradeID using best-match deduplication
SELECT TradeID, AccountID, SecurityID, TradeDate, SettleDate,
       Side, Price, Quantity, Amount
FROM (
    SELECT TradeID, AccountID, SecurityID, TradeDate, SettleDate,
           Side, Price, Quantity, Amount,
           ROW_NUMBER() OVER (
               PARTITION BY TradeID
               ORDER BY
                   ABS(CAST(Amount AS DECIMAL(18,2)) -
                       ROUND(CAST(Price AS DECIMAL(18,4)) *
                           CAST(Quantity AS DECIMAL(18,4)), 2)
                   ) ASC
           ) AS rn
    FROM (
        SELECT DISTINCT TradeID, AccountID, SecurityID,
                        TradeDate, SettleDate, Side, Price, Quantity, Amount
        FROM stg.stg_trades
    ) deduped
    ) ranked
WHERE rn = 1
) src
INNER JOIN dwh.dim_account da
  ON da.account_id = src.AccountID AND da.is_current = 'Y'
INNER JOIN dwh.dim_security ds
  ON ds.security_id = src.SecurityID AND ds.is_active = 'Y'
INNER JOIN dwh.dim_date dd_trade
  ON dd_trade.full_date = CAST(src.TradeDate AS DATE)
INNER JOIN dwh.dim_date dd_settle
  ON dd_settle.full_date = CAST(src.SettleDate AS DATE);
END;

```

SP-8: dwh.usp_load_fact_cashflow

The is_inflow flag is derived inline using a CASE expression on the Amount column: positive values are flagged 'Y' (inflow), negative values are flagged 'N' (outflow). This derived column simplifies Power BI filtering and DAX measure construction.

```

CREATE PROCEDURE dwh.usp_load_fact_cashflow
AS
BEGIN
    SET NOCOUNT ON;
    TRUNCATE TABLE dwh.fact_cashflow;

    INSERT INTO dwh.fact_cashflow (
        account_sk, date_sk, cashflow_id, account_id,
        cashflow_date, cashflow_type, amount, is_inflow
    )
    SELECT
        da.account_sk, dd.date_sk,
        src.CashflowID, src.AccountID,
        CAST(src.CashflowDate AS DATE),
        src.Type,
        CAST(src.Amount AS DECIMAL(18,2)),
        CASE WHEN CAST(src.Amount AS DECIMAL(18,2)) > 0 THEN 'Y' ELSE 'N' END
    FROM stg.stg_cashflows src
    INNER JOIN dwh.dim_account da

```

```

        ON da.account_id = src.AccountID AND da.is_current = 'Y'
    INNER JOIN dwh.dim_date dd
        ON dd.full_date = CAST(src.CashflowDate AS DATE);
END;

```

SP-9: dwh.usp_load_fact_fee

```

CREATE PROCEDURE dwh.usp_load_fact_fee
AS
BEGIN
    SET NOCOUNT ON;
    TRUNCATE TABLE dwh.fact_fee;

    INSERT INTO dwh.fact_fee (
        account_sk, date_sk, fee_id, account_id,
        fee_date, fee_type, amount
    )
    SELECT
        da.account_sk, dd.date_sk,
        src.FeeID, src.AccountID,
        CAST(src.FeeDate AS DATE),
        src.FeeType,
        CAST(src.Amount AS DECIMAL(18,2))
    FROM stg.stg_fees src
    INNER JOIN dwh.dim_account da
        ON da.account_id = src.AccountID AND da.is_current = 'Y'
    INNER JOIN dwh.dim_date dd
        ON dd.full_date = CAST(src.FeeDate AS DATE);
END;

```

SP-10: dwh.usp_load_fact_security_price

ClosePrice handling uses CASE WHEN src.ClosePrice = '' OR src.ClosePrice IS NULL THEN NULL ELSE CAST(...) END. This explicitly converts both empty strings and NULL source values into SQL NULL, ensuring consistent representation and preventing CAST errors on blank values.

```

CREATE PROCEDURE dwh.usp_load_fact_security_price
AS
BEGIN
    SET NOCOUNT ON;
    TRUNCATE TABLE dwh.fact_security_price;

    INSERT INTO dwh.fact_security_price (
        security_sk, date_sk, security_id, price_date, close_price
    )
    SELECT
        ds.security_sk, dd.date_sk,
        src.SecurityID,
        CAST(src.PriceDate AS DATE),
        CASE WHEN src.ClosePrice = '' OR src.ClosePrice IS NULL
            THEN NULL
            ELSE CAST(src.ClosePrice AS DECIMAL(18,4)) END
    FROM stg.stg_security src
    INNER JOIN dwh.dim_security ds
        ON ds.security_id = src.SecurityID AND ds.is_current = 'Y'
    INNER JOIN dwh.dim_date dd
        ON dd.full_date = CAST(src.PriceDate AS DATE);
END;

```

```

FROM stg.stg_prices src
INNER JOIN dwh.dim_security ds
    ON ds.security_id = src.SecurityID AND ds.is_active = 'Y'
INNER JOIN dwh.dim_date dd
    ON dd.full_date = CAST(src.PriceDate AS DATE);
END;

```

SP-11: dwh.usp_load_fact_performance

ReturnPct is handled with the same NULL/blank pattern as ClosePrice. The 109 records with missing ReturnPct are loaded with explicit NULL, while all other percentage fields are cast to DECIMAL(10,6). The SP joins against dim_account (is_current = 'Y'), dim_benchmark (is_active = 'Y'), and dim_date.

```

CREATE PROCEDURE dwh.usp_load_fact_performance
AS
BEGIN
    SET NOCOUNT ON;
    TRUNCATE TABLE dwh.fact_performance;

    INSERT INTO dwh.fact_performance (
        account_sk, benchmark_sk, date_sk,
        account_id, benchmark_code, month_start,
        return_pct, benchmark_return_pct, excess_return_pct, cumulative_return_pct
    )
    SELECT
        da.account_sk, db.benchmark_sk, dd.date_sk,
        src.AccountID, src.BenchmarkCode,
        CAST(src.MonthStart AS DATE),
        CASE WHEN src.ReturnPct = '' OR src.ReturnPct IS NULL
            THEN NULL
            ELSE CAST(src.ReturnPct AS DECIMAL(10,6)) END,
        CAST(src.BenchmarkReturnPct AS DECIMAL(10,6)),
        CAST(src.ExcessReturnPct AS DECIMAL(10,6)),
        CAST(src.CumulativeReturnPct AS DECIMAL(10,6))
    FROM stg.stg_performance src
    INNER JOIN dwh.dim_account da
        ON da.account_id = src.AccountID AND da.is_current = 'Y'
    INNER JOIN dwh.dim_benchmark db
        ON db.benchmark_code = src.BenchmarkCode AND db.is_active = 'Y'
    INNER JOIN dwh.dim_date dd
        ON dd.full_date = CAST(src.MonthStart AS DATE);
END;

```

8. Pipeline Validation and Final Output Verification

After each complete pipeline run, row count validation queries were executed against both the staging and data warehouse layers to confirm that all records were loaded and transformed correctly. The validation results confirmed that actual row counts matched expected values across all twenty-two tables.

8.1 Final Row Count Verification

```
SELECT 'dim_advisor'           , COUNT(*) FROM dwh.dim_advisor      UNION ALL
SELECT 'dim_benchmark'        , COUNT(*) FROM dwh.dim_benchmark  UNION ALL
SELECT 'dim_security'         , COUNT(*) FROM dwh.dim_security   UNION ALL
SELECT 'dim_client'           , COUNT(*) FROM dwh.dim_client      UNION ALL
SELECT 'dim_account'          , COUNT(*) FROM dwh.dim_account    UNION ALL
SELECT 'fact_holding'         , COUNT(*) FROM dwh.fact_holding    UNION ALL
SELECT 'fact_trade'           , COUNT(*) FROM dwh.fact_trade      UNION ALL
SELECT 'fact_cashflow'        , COUNT(*) FROM dwh.fact_cashflow  UNION ALL
SELECT 'fact_fee'             , COUNT(*) FROM dwh.fact_fee        UNION ALL
SELECT 'fact_security_price'   , COUNT(*) FROM dwh.fact_security_price UNION ALL
SELECT 'fact_performance'     , COUNT(*) FROM dwh.fact_performance;
```

Table	Layer	Expected Rows	Verification Notes
stg.stg_advisors	Staging	350	Matches source CSV exactly
stg.stg_clients	Staging	8,000	Matches source CSV exactly
stg.stg_accounts	Staging	12,000	Matches source CSV exactly
stg.stg_securities	Staging	3,000	Matches source CSV exactly
stg.stg_benchmarks	Staging	4,020	Matches source CSV exactly
stg.stg_cashflows	Staging	30,000	Matches source CSV exactly
stg.stg_fees	Staging	18,000	Matches source CSV exactly
stg.stg_holdings	Staging	96,027	Matches source CSV exactly
stg.stg_performance	Staging	110,000	Matches source CSV exactly
stg.stg_prices	Staging	301,500	Matches source CSV exactly
stg.stg_trades	Staging	100,400	Matches source CSV exactly — duplicates resolved in SP
dwh.dim_advisor	DWH	350	Clean — 1 region defaulted to 'Unknown'

Table	Layer	Expected Rows	Verification Notes
dwh.dim_benchmark	DWH	6	6 distinct benchmark codes from 4,020 source rows
dwh.dim_security	DWH	3,000	Clean — 672 sectors defaulted to 'N/A'
dwh.dim_client	DWH	8,000	SCD Type 2 — all current records (is_current='Y')
dwh.dim_account	DWH	12,000	SCD Type 2 — all current records (is_current='Y')
dwh.fact_holding	DWH	96,027	AUM snapshot — valuation date 2025-10-31
dwh.fact_trade	DWH	100,000	400 exact duplicates removed; 3 amended amounts resolved
dwh.fact_cashflow	DWH	30,000	Signed amounts loaded — is_inflow flag derived
dwh.fact_fee	DWH	18,000	All positive fee amounts loaded
dwh.fact_security_price	DWH	301,022	478 records with NULL close_price loaded
dwh.fact_performance	DWH	110,000	109 records with NULL return_pct loaded as NULL

8.2 Data Quality Summary

Issue Category	Affected Table(s)	Count	Resolution Applied
NULL descriptor — RiskProfile	stg_clients → dim_client	66 rows	Defaulted to 'Unknown' via ISNULL(NULLIF())
NULL descriptor — Region	stg_advisors → dim_advisor	1 row	Defaulted to 'Unknown' via ISNULL(NULLIF())
NULL descriptor — Sector	stg_securities → dim_security	672 rows	Defaulted to 'N/A' via ISNULL(NULLIF())
NULL measure — ReturnPct	stg_performance → fact_performance	109 rows	Loaded as explicit NULL; not dropped
NULL measure — ClosePrice	stg_prices → fact_security_price	478 rows	Loaded as explicit NULL; not dropped
Exact duplicate rows	stg_trades → fact_trade	400 rows	Removed via SELECT DISTINCT in SP-7
Amended trade amounts	stg_trades → fact_trade	3 TradeIDs	Best-match row kept via ROW_NUMBER() + ABS() logic

8.2 Referential Integrity Check

```
SELECT 'fact_holding orphan accounts' , COUNT(*) FROM dwh.fact_holding WHERE account_sk
IS NULL UNION ALL
SELECT 'fact_holding orphan securities', COUNT(*) FROM dwh.fact_holding WHERE security_sk
IS NULL UNION ALL
SELECT 'fact_trade orphan accounts' , COUNT(*) FROM dwh.fact_trade WHERE account_sk
IS NULL UNION ALL
SELECT 'fact_trade orphan securities' , COUNT(*) FROM dwh.fact_trade WHERE security_sk
IS NULL UNION ALL
SELECT 'fact_cashflow orphan accounts' , COUNT(*) FROM dwh.fact_cashflow WHERE account_sk
IS NULL UNION ALL
SELECT 'fact_fee orphan accounts' , COUNT(*) FROM dwh.fact_fee WHERE account_sk
IS NULL UNION ALL
SELECT 'fact_price orphan securities' , COUNT(*) FROM dwh.fact_security_price WHERE security_sk
IS NULL UNION ALL
SELECT 'fact_perf orphan accounts' , COUNT(*) FROM dwh.fact_performance WHERE account_sk
IS NULL UNION ALL
SELECT 'fact_perf orphan benchmarks' , COUNT(*) FROM dwh.fact_performance WHERE benchmark_sk
IS NULL;
```

Result: ALL checks returned 0 — zero orphan records across all 6 fact tables.

8.3 Duplicate Trade Check

```
SELECT COUNT(*) AS duplicate_trade_ids
FROM (
    SELECT trade_id FROM dwh.fact_trade
    GROUP BY trade_id HAVING COUNT(*) > 1
) x;
```

Result: 0 — zero duplicate TradeIDs after SP fix.

8.4 Audit Log Check

9. Analytics Readiness and Power BI Integration

The data warehouse is fully compatible with Microsoft Power BI and has been designed specifically to support the analytical requirements of FinWealth Global's business intelligence layer. The Star Schema structure — with clean dimension tables, resolved surrogate keys, and validated fact table measures — allows Power BI to establish relationships, build time intelligence calculations, and render complex dashboards without requiring additional transformation in Power Query.

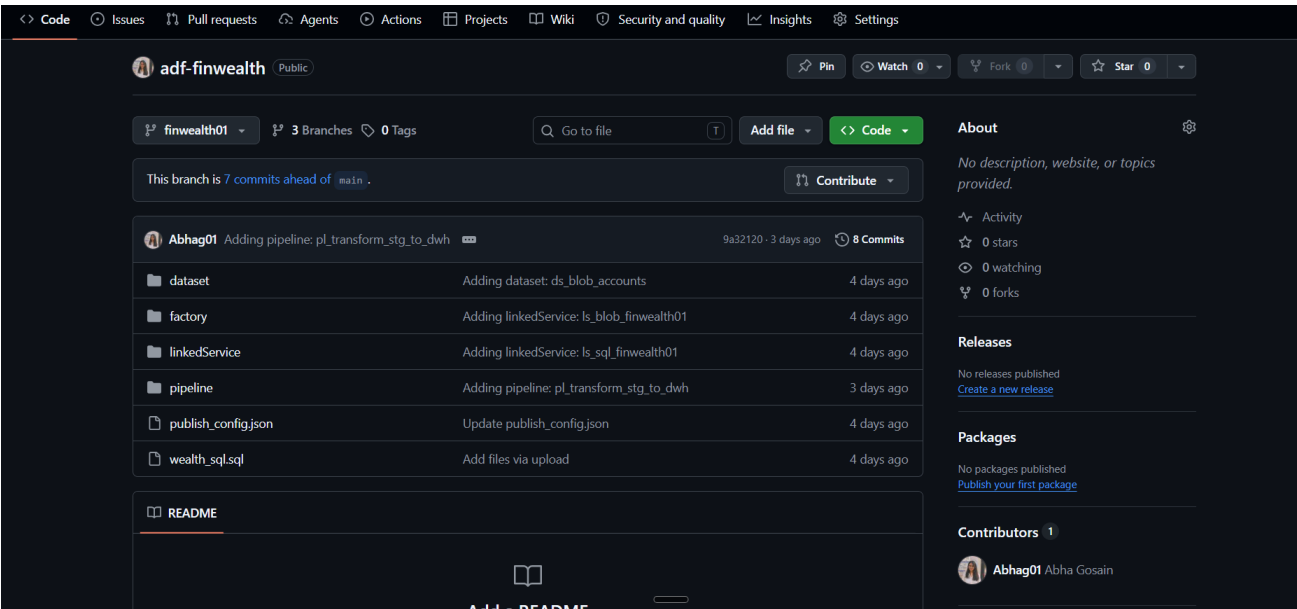
9.1 Power BI Data Model

Relationships in the Power BI data model are established between Dim_Date and each fact table using the full_date column, and between each dimension table and the relevant fact tables using surrogate key columns. The Dim_Date table was marked as a Date Table in Power BI using the full_date column, enabling native time intelligence functions.

9.2 Supported Analytical Use Cases

- AUM analysis by client segment, advisor region, account type, and asset class using Fact_Holdings.market_value.
- Advisor performance benchmarking using Fact_Performance.return_pct versus benchmark_return_pct and excess_return_pct.
- Trade activity analysis by security, account, side (BUY/SELL), and settlement efficiency using Fact_Trades.settle_lag_days.
- Fee revenue analysis by fee type, account, and time period using Fact_Fees.
- Cashflow analysis distinguishing client inflows from outflows using Fact_Cashflows.is_inflow.
- Security price trend analysis using Fact_Security_Price.close_price across the Dim_Date time dimension.
- Time intelligence calculations including Year-to-Date (YTD), Month-to-Date (MTD), and Quarter-to-Date (QTD) using Power BI's native time intelligence functions against the Dim_Date table.

9.3 GitHub Integration and Version Control



The ADF instance (adf-finwealth01) was connected to a GitHub repository (Abhag01/adf-finwealth) through the ADF Studio Git configuration. This integration ensures that all pipeline definitions, dataset configurations, and linked service definitions are automatically committed to the repository whenever changes are published in ADF Studio. This provides a full version history of all pipeline changes and enables collaborative development.

Repository Folder	Contents
dataset/	22 JSON files — one per ADF dataset (11 blob + 11 SQL staging)
linkedService/	3 JSON files — ls_blob_finwealth_raw, ls_sql_finwealth_dwh, ls_keyvault_finwealth
pipeline/	3 JSON files — pl_ingest_AllSources, pl_transform_stg_to_dwh, pl_masterpipeline
factory/	1 JSON file — ADF factory-level configuration

10. Conclusion

This project successfully delivered a production-grade, end-to-end data engineering pipeline for FinWealth Global on the Microsoft Azure platform. Starting from eleven raw CSV source files totalling over 683,000 source rows across all files, the pipeline ingests, validates, transforms, and loads data into a well-structured Star Schema data warehouse that is fully ready for Power BI analytics.

Key technical achievements of this project include:

- Design and implementation of a three-layer data architecture (Blob Storage → Staging → Data Warehouse) using Azure-native services.
- Eleven parallel Copy Activities in `pl_ingest_AllSources` providing efficient, full-refresh ingestion of all source files.
- SCD Type 2 implementation for `Dim_Client` and `Dim_Account`, preserving complete historical records for regulatory compliance.
- SCD Type 1 implementation for `Dim_Advisor`, `Dim_Security`, and `Dim_Benchmark` with proper null handling using `ISNULL(NULLIF())` patterns.
- Advanced trade deduplication logic in `usp_load_fact_trade` using `ROW_NUMBER()` with best-match amount resolution.
- Surrogate key strategy using `IDENTITY(1,1)` columns decoupled from source natural keys, with foreign key constraints enforcing referential integrity.
- Audit and error logging infrastructure in `dbo.audit_pipeline_log` tracking every pipeline run.
- Secure credential management via Azure Key Vault, with no sensitive values exposed in pipeline definitions or version control.
- Full GitHub integration providing complete version history of all ADF pipeline JSON definitions and configurations.

The resulting data warehouse — comprising six dimension tables including `dim_date` and six fact tables spanning over 667,000 DWH rows — provides FinWealth Global with a reliable, scalable, and analytics-ready foundation for AUM reporting, advisor performance tracking, trade activity analysis, and client profitability measurement.

All pipeline runs completed successfully with row counts matching expected values across all twenty-two staging and data warehouse tables. The pipeline is designed for repeatability and can be re-executed on a scheduled basis to refresh the data warehouse as new source files are made available.

— End of Document —